

我帮你把 **Shell 核心知识点**整理成一份清晰、实用的知识框架，适合**快速查阅和实战**，而不是零散记笔记。

一、Shell 是什么

- Shell** 是命令解释器，负责接收用户输入的命令并调用系统内核执行
- 常见类型：`bash`（最常用）、`zsh`、`sh`、`fish`
- 脚本文件通常以 `#!/bin/bash` 开头（称为 shebang），指定用什么解释器执行

二、基础语法（必会）

1. 变量

```
name="John"      # 定义（等号两边不能有空格）
echo $name       # 使用（加 $）
echo ${name}     # 推荐加花括号，更清晰
readonly age=25  # 只读变量
unset name       # 删除变量
```

2. 特殊变量

变量	含义
<code>\$0</code>	脚本名称
<code>\$1-\$9</code>	第 1-9 个参数
<code>\$#</code>	参数个数

变量	含义
<code>\$@</code>	所有参数（分开）
<code>\$*</code>	所有参数（合并）
<code> \$? </code>	上一条命令的退出状态（0 成功，非 0 失败）
<code> \$\$ </code>	当前进程 PID

3. 字符串

```
str="hello"
echo ${#str}      # 长度：5
echo ${str:1:3}   # 截取：ell
```

4. 数组

```
arr=(a b c)
echo ${arr[0]}    # 第一个元素
echo ${arr[@]}    # 所有元素
echo ${#arr[@]}   # 元素个数
```

三、流程控制

1. 条件判断 `if`

```
if [ condition ]; then
    command
elif [ condition2 ]; then
    command
else
    command
fi
```

常用条件写法：

```
[ "$a" == "$b" ]    # 字符串相等
[ "$a" != "$b" ]    # 字符串不等
[ -z "$a" ]         # 字符串为空
[ -n "$a" ]         # 字符串非空
[ $a -eq $b ]       # 数字相等 (-ne, -gt, -lt, -ge, -le)
[ -f "file" ]       # 是普通文件
[ -d "dir" ]        # 是目录
[ -e "path" ]       # 路径存在
[ -r "file" ]       # 可读 (-w 写, -x 执行)
```

双中括号更强大（推荐）：

```
[[ "$a" == *"hello"* ]] # 支持通配符
[[ "$a" =~ ^[0-9]+$ ]]  # 支持正则
```

2. 循环

```
# for 循环（遍历列表）
for i in 1 2 3 4 5; do
    echo $i
done

for file in *.txt; do
```

```
    echo $file
done

# C 风格 for
for ((i=0; i<10; i++)); do
    echo $i
done

# while 循环
while [ $count -lt 5 ]; do
    echo $count
    ((count++))
done

# until 循环 (条件为假时执行)
until [ $count -ge 5 ]; do
    echo $count
    ((count++))
done
```

3. case 分支

```
case $var in
    start)
        echo "starting"
        ;;
    stop)
        echo "stopping"
        ;;
    *)
        echo "unknown"
        ;;
esac
```

四、函数

```
# 定义函数
my_func() {
    echo "参数1: $1"
    echo "参数2: $2"
    return 0      # 返回状态码 (0-255)
}

# 调用
my_func "hello" "world"

# 获取函数返回值
my_func
result=$?
```

注意:

- 函数内变量默认是全局的, 用 `local` 声明局部变量
- 函数返回值只能用 `return` 返回数字, 想返回字符串用 `echo` 捕获

```
get_name() {
    echo "John"
}
name=$(get_name)
```

五、重定向与管道 (非常重要)

符号	含义
----	----

>	覆盖写入文件
>>	追加写入文件
<	从文件读入
2>	错误输出重定向
&>	正确和错误都重定向
2>&1	把错误输出到正确输出的位置
\	管道，前一个命令的输出作为后一个命令的输入

示例：

```
echo "hello" > log.txt      # 写入
echo "world" >> log.txt    # 追加
python app.py > out.log 2>&1 # 所有输出都写入文件
ls | grep ".txt"          # 管道
```

六、后台运行（你刚才问的）

命令/符号	作用
command &	后台运行（当前终端）
jobs	查看当前终端的后台任务
fg %编号	切换到前台
bg %编号	切换到后台继续运行
Ctrl+Z	暂停当前前台任务

命令/符号	作用
<code>nohup command &</code>	后台运行，关掉终端也不停
<code>disown</code>	把任务从当前 shell 剥离

七、常用工具命令

命令	用途
<code>grep</code>	文本搜索
<code>sed</code>	文本替换/编辑
<code>awk</code>	文本处理/格式化
<code>find</code>	查找文件
<code>xargs</code>	将标准输入转为命令行参数
<code>cut</code>	截取列
<code>sort</code>	排序
<code>uniq</code>	去重
<code>wc</code>	统计行/字/字节
<code>head/tail</code>	看文件头/尾

示例：

```
grep "error" log.txt
find . -name "*.py" | xargs grep "def"
cat file | awk '{print $1, $3}'
ps aux | grep python | awk '{print $2}' | xargs kill
```

八、常用脚本写法（拿来即用）

1. 脚本模板

```
#!/bin/bash
set -e      # 遇到错误立即退出
set -u      # 使用未定义变量时报错
set -o pipefail # 管道中任何一个命令失败都算失败

echo "脚本开始..."
# 你的代码
echo "脚本结束"
```

2. 判断命令是否存在

```
if command -v python &> /dev/null; then
    echo "Python 已安装"
else
    echo "请安装 Python"
fi
```

3. 循环读取文件每行


```
while IFS= read -r line; do
    echo "$line"
done < file.txt
```

4. 获取脚本所在目录

```
SCRIPT_DIR=$(cd "$(dirname "$0")" && pwd)
```

5. 颜色输出

```
RED='\033[0;31m'
GREEN='\033[0;32m'
NC='\033[0m'      # No Color
echo -e "${RED}错误${NC}"
```

九、调试技巧

```
bash -x script.sh    # 打印每条执行的命令
bash -n script.sh    # 仅检查语法，不执行
```

在脚本内加 `set -x` 开启调试，`set +x` 关闭。

十、注意事项（易错点）

错误写法	正确写法	原因
<code>name = "John"</code>	<code>name="John"</code>	等号两边不能有空格
<code>if [\$a -eq 1]</code>	<code>if [\$a -eq 1]</code>	方括号前后必须有空格

错误写法	正确写法	原因
<code>echo \$name</code>	<code>echo "\$name"</code>	变量加引号防止空格分割
<code>for i in \$(ls)</code>	<code>for i in *.txt</code>	避免解析 ls 输出
反引号 <code>`cmd`</code>	<code>\$(cmd)</code>	推荐用 <code>\$()</code> ，更清晰且可嵌套